Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 01. Patrones de diseño creacionales

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Abstract Factory

- *Abstract Factory*
- Prototype

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Abstract Factory

- Propósito
  - Proporciona una interfaz para crear familias de objetos relacionados o dependientes, sin especificar sus clases concretas.
- Problema / Motivación
  - Un hospital necesita crear diferentes tipos de personal, por ejemplo Médicos y Colaboradores.
  - Existen además distintas modalidades de contratación:

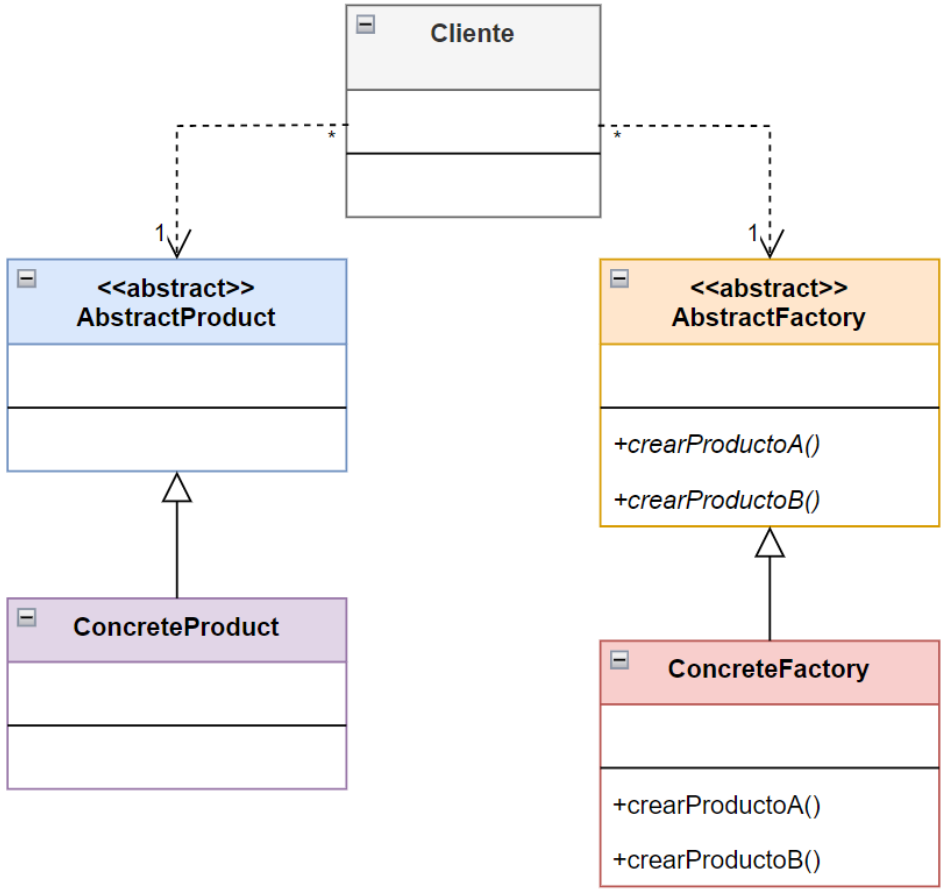
    contrato fijo;

    contrato temporal.
  - Los objetos creados deben pertenecer a una misma familia de contratación. Por ejemplo, una configuración puede trabajar con personal fijo y otra con personal temporal.
  - El sistema debe poder cambiar de una familia a otra sin que el cliente dependa directamente de las clases concretas que instancia.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Abstract Factory

Estructura



- Participantes
  - AbstractFactory:

    Declara las operaciones necesarias para crear los diferentes productos de una familia.
  - ConcreteFactory:

    Implementa las operaciones de creación y produce una familia concreta de productos compatibles.
  - AbstractProduct:

    Define el contrato común para cada tipo de producto que forma parte de la familia.
  - ConcreteProduct:

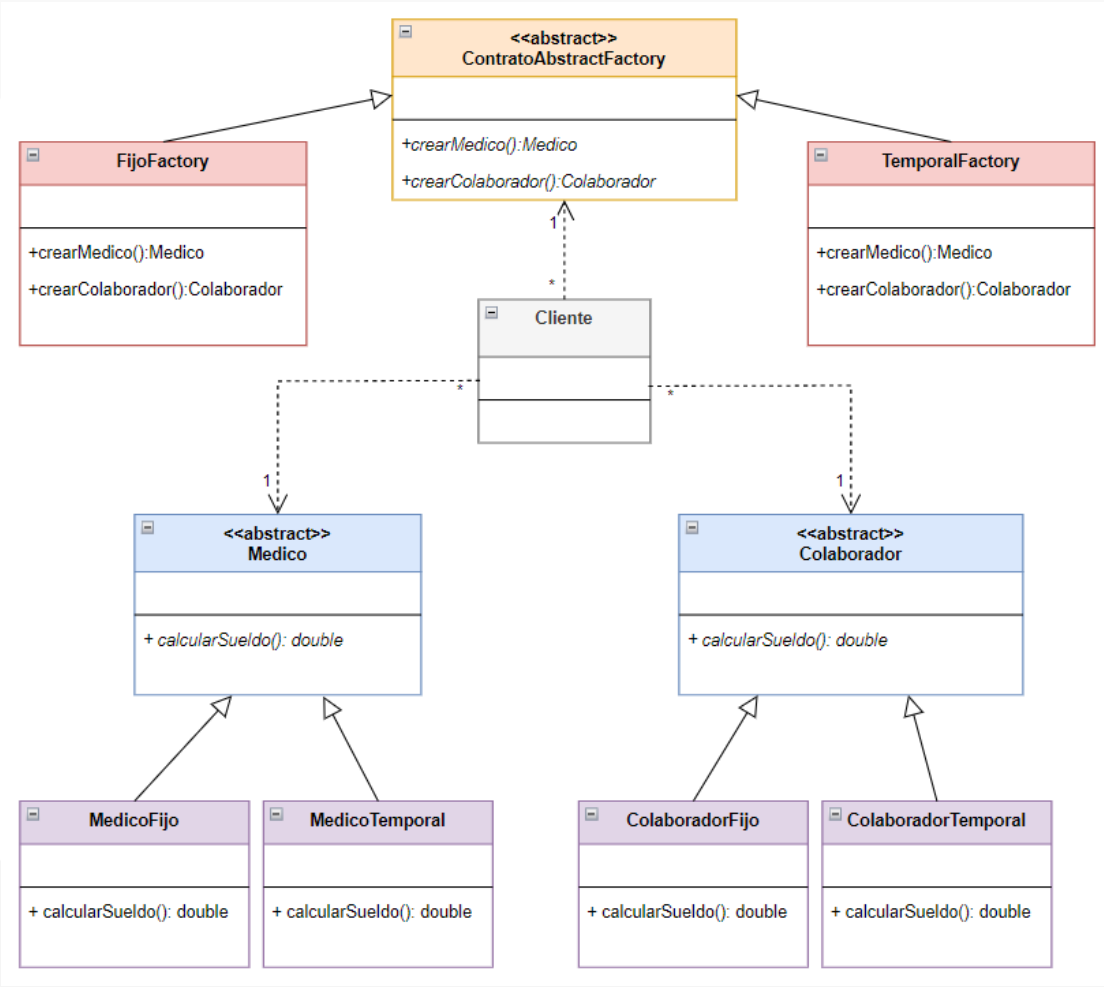
    Implementa un producto concreto perteneciente a una familia específica.
  - Cliente:

    Trabaja únicamente con las abstracciones AbstractFactory y AbstractProduct.
- Colaboraciones
  - El cliente selecciona una ConcreteFactory. La fábrica concreta crea los productos pertenecientes a una misma familia.
  - El cliente utiliza los productos mediante sus abstracciones.
  - Para cambiar de familia, el cliente utiliza otra ConcreteFactory sin modificar su lógica principal.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Abstract Factory

Solución



https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/abstractFactory

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Abstract Factory

- Aplicabilidad
  - El sistema debe ser independiente de cómo se crean y representan los productos concretos.
  - El sistema debe poder configurarse con una familia de productos entre varias alternativas.
  - Los productos de una misma familia están diseñados para utilizarse conjuntamente y debe mantenerse su compatibilidad.
  - Se desea trabajar únicamente con las abstracciones de los productos, ocultando sus implementaciones concretas.
  - Se necesita cambiar una familia completa de productos sin modificar la lógica principal del cliente.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Actividad en clase: ¿Abstract Factory?

Caso 1 — Interfaz gráfica multiplataforma

Una aplicación debe ejecutarse en Windows y macOS. Cada plataforma utiliza botones, menús y cuadros de diálogo con apariencia y comportamiento propios.

¿Cómo diseñarían la creación de los componentes para cambiar de plataforma sin modificar el código cliente y manteniendo una familia visual coherente?

Caso 2 — Equipamiento para videojuegos

Un videojuego permite seleccionar diferentes mundos: medieval y futurista. Cada mundo dispone de armas, armaduras y vehículos propios.

¿Cómo diseñarían la creación de los objetos para garantizar que todos pertenezcan al mismo mundo seleccionado?

Caso 3 — Servicios en la nube

Una aplicación puede desplegarse en AWS o Azure. Para cada proveedor necesita almacenamiento, mensajería y servicios de autenticación compatibles con su plataforma.

¿Cómo diseñarían el sistema para cambiar de proveedor creando una familia completa de servicios sin acoplar el cliente a implementaciones concretas?

Caso 4 — Mobiliario de oficina

Una empresa ofrece líneas de mobiliario Clásica y Moderna. Cada línea incluye escritorio, silla y archivador diseñados para utilizarse conjuntamente.

¿Cómo diseñarían la creación de una oficina completa garantizando que todos los muebles pertenezcan a la misma familia?

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Tema:

Prototype

- Abstract Factory
- *Prototype*

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Prototype

- Propósito
  - Especifica los tipos de objetos a crear mediante una instancia prototípica y crea nuevos objetos copiando ese prototipo.
- Problema / Motivación
  - Una aplicación necesita crear varios objetos que comparten una configuración inicial compleja.
  - Por ejemplo, los jugadores de la selección de Ecuador comparten determinados valores:

    nacionalidad: ecuatoriana;

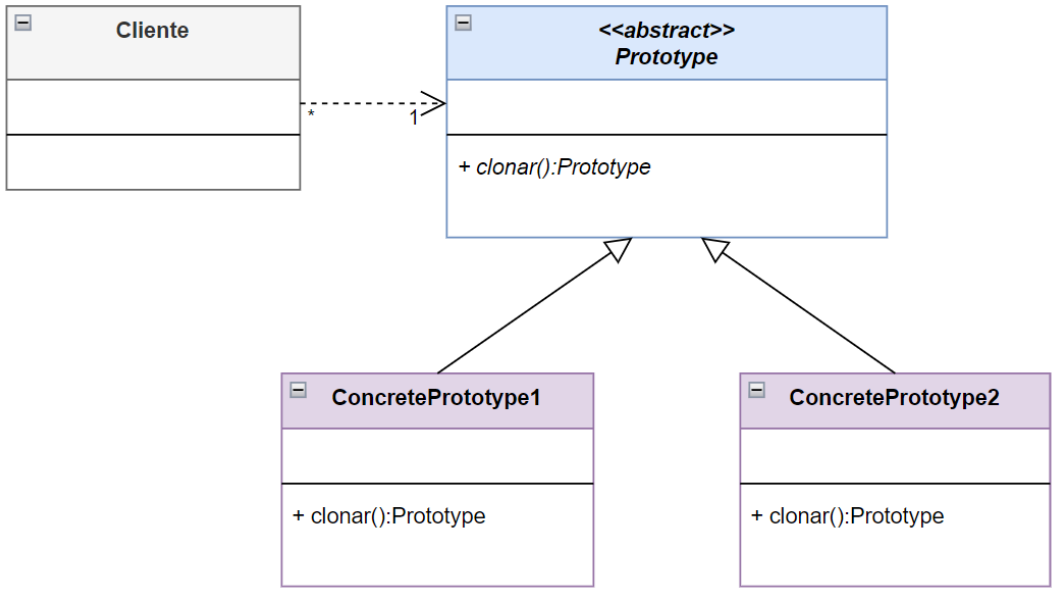
    color de camiseta: amarillo;

    selección: Ecuador.
  - En lugar de configurar nuevamente esos atributos para cada jugador, podemos crear un objeto Jugador previamente configurado y utilizarlo como prototipo.
  - Cada nuevo jugador se obtiene copiando ese prototipo y posteriormente se personalizan los atributos que cambian, como nombre, número o posición.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Prototype

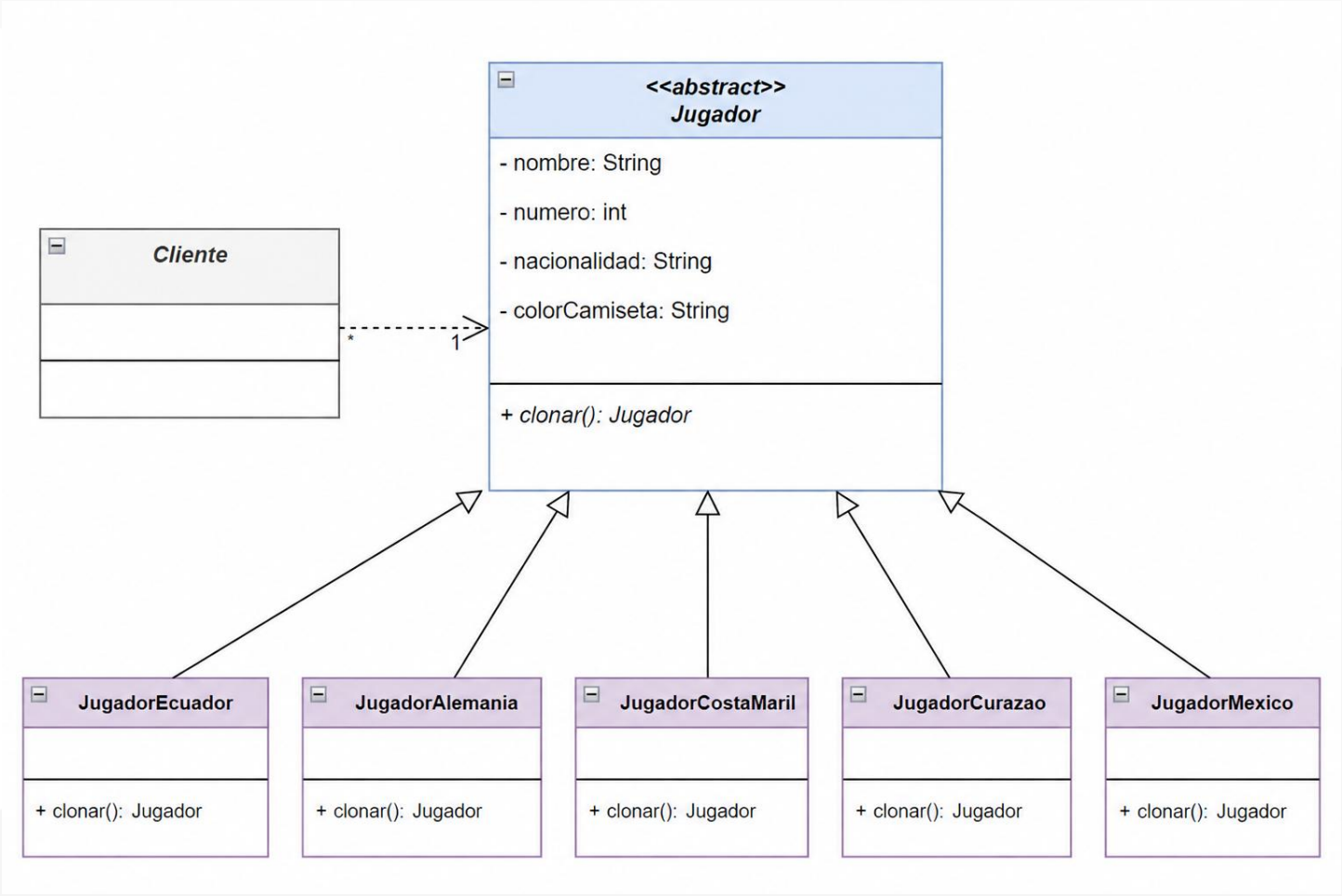
Estructura



- Participantes
  - Prototype
    Declara la operación que permite copiar una instancia existente.
  - ConcretePrototype:
    Implementa la operación de copia y define cómo se duplica su estado.
  - Cliente:
    Obtiene un prototipo existente y solicita una copia cuando necesita crear un nuevo objeto.
- Colaboraciones
  - El cliente trabaja con la abstracción Prototype.
  - Solicita una nueva instancia mediante una operación como clonar().
  - El ConcretePrototype crea la copia utilizando como punto de partida su propio estado.
  - La nueva instancia puede modificarse posteriormente sin alterar el prototipo original.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Prototype



https://github.com/mortizo/pds-unidad01-ooms/tree/master/src/main/java/p69/prototype

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Prototype

- Aplicabilidad
  - Las clases concretas que deben crearse se determinan o configuran en tiempo de ejecución.
  - Se desea evitar una jerarquía de fábricas paralela a la jerarquía de productos.
  - Existen varias configuraciones o estados iniciales recurrentes que pueden mantenerse como prototipos.
  - Resulta más conveniente copiar un objeto previamente configurado que reconstruir su estado desde cero.
  - La creación del objeto implica una configuración compleja que puede reutilizarse como punto de partida para nuevas instancias.
- Precaución
  - La estrategia de copia debe definir correctamente qué estado se duplica y qué referencias pueden compartirse, especialmente cuando existen objetos internos mutables

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Prototype

Actividad en clase: ¿Prototype?

Caso 1 — Personajes de videojuego

Un videojuego dispone de personajes con configuraciones base previamente definidas: guerrero, mago y arquero.

Cada configuración incluye atributos, habilidades y equipamiento inicial.

¿Cómo crearían nuevos personajes a partir de configuraciones existentes sin reconstruir cada objeto desde cero?

Caso 2 — Plantillas de documentos

Una organización utiliza documentos con estructuras preconfiguradas para contratos, informes y memorandos.

Cada plantilla contiene encabezados, secciones, estilos y datos comunes.

¿Cómo permitirían crear nuevos documentos utilizando una plantilla existente como punto de partida?

Caso 3 — Configuración de servidores

Una plataforma administra configuraciones estándar de servidores para desarrollo, pruebas y producción.

Cada configuración contiene parámetros de red, recursos y servicios habilitados.

¿Cómo crearían nuevas configuraciones basándose en una configuración existente y modificando únicamente los valores necesarios?

Caso 4 — Diseño de productos personalizados

Una tienda ofrece bicicletas con configuraciones base: urbana, montaña y ruta.

Cada modelo tiene componentes previamente definidos, pero el cliente puede modificar ciertos elementos.

¿Cómo diseñarían la creación de nuevas bicicletas partiendo de configuraciones existentes sin reconstruir todos sus componentes?

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Unidad 01.04

Retroalimentación y cierre

Qué vimos hoy

- Abstract Factory: permite crear familias de objetos relacionados o compatibles sin depender de sus clases concretas.
- Prototype: permite crear nuevos objetos tomando como punto de partida una instancia existente.
- Abstract Factory resulta útil cuando necesitamos cambiar una familia completa de productos.
- Prototype resulta útil cuando una configuración existente puede reutilizarse mediante copia.
- Ambos patrones desacoplan al cliente de detalles concretos de creación, pero resuelven problemas diferentes.
- La selección de un patrón debe justificarse por el problema de diseño, no por la posibilidad de implementarlo.

Antes de finalizar:

- ¿Cuándo utilizarían Abstract Factory en lugar de Factory Method?
- ¿Cuándo preferirían Prototype en lugar de construir nuevamente un objeto?
- ¿Qué riesgo existe al copiar objetos que contienen referencias a otros objetos?

Idea final: Abstract Factory reutiliza una estructura de creación para producir familias coherentes; Prototype reutiliza el estado de una instancia existente para crear nuevos objetos.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.
Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.
Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.
Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.
Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.
IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.
Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Anexos

Links de Referencia

- https://www.campusmvp.es/recursos/post/los-conceptos-fundamentales-sobre-programacion-orientada-objetos-explicados-de-manera-simple.aspx
- https://codideep.com/blogpost/stupid-vs-solid
- https://refactoring.guru/design-patterns
- https://reactiveprogramming.io/books/design-patterns/es

Este material es de autoría y uso académico exclusivo de
Mauricio Ortiz Ochoa

Gracias por su atención

CC BY NC ND